Université Sidi Mohamed Ben Abdellah

École Nationale des Sciences Appliquées de Fès

Filière Génie du Développement Numérique et Cybersécurité (GDNC)

Rapport de mini-projet, module DevSecOps

Sécurisation des workloads Kubernetes selon une approche Zero Trust

---

Conception, mise en œuvre et mesure des résultats

Réalisé par :

Abir Majdi
Doha El Merabet

Encadré par :

Pr. Mohammed Airaj

Année universitaire 2026–2027

Remis le 9 novembre 2026

Table des matières

1 Introduction **1**

2 Architecture et environnement de test **1**

2.1 Application et composants 1

2.2 Frontières de confiance 2

2.3 Démarche et organisation du travail 3

3 Partie A : identités, réseau et admission **3**

3.1 Comptes de service et RBAC 3

3.2 Segmentation réseau avec Cilium 3

3.3 Contrôle d'admission 4

3.4 Mesure automatisée et reproductibilité 5

4 Partie B : secrets, traçabilité et détection **6**

4.1 Gestion des secrets avec Vault et External Secrets Operator 6

4.2 Audit de l'API et chiffrement au repos 7

4.3 Détection à l'exécution avec Falco 7

4.4 Analyse des images et seconde reconstruction 8

5 Résultats et évaluation **8**

5.1 Réduction du rayon d'action 8

5.2 Conformité et coût 9

5.3 Sécurité du stockage 9

5.4 Limites 9

6 Réponses aux questions d'analyse du sujet **10**

7 Discussion : justification de chaque outil **10**

8 Conclusion **11**

1. Introduction

Les clusters Kubernetes partagés entre plusieurs applications posent un problème de confiance implicite. Par défaut, tout pod peut joindre tout autre pod, chaque compte de service reçoit un jeton monté automatiquement, et le serveur d'API accepte des conteneurs privilégiés tant qu'aucune règle d'admission ne s'y oppose. Une seule application compromise suffit alors à atteindre le reste du cluster.

Le modèle Zero Trust, formalisé par le NIST [1], part du principe inverse : aucune communication ni aucun accès n'est accordé sur la seule base de la position dans le réseau, et chaque autorisation doit être explicite. Le sujet de ce mini-projet pose la question suivante :

> *Comment appliquer le principe Zero Trust aux workloads Kubernetes sans multiplier inutilement les mécanismes de sécurité ?*

Nous avons choisi d'y répondre par la mesure. L'application est d'abord déployée sans aucune protection, puis quatre scénarios d'attaque sont rejoués (tableau 1). Les couches de sécurité sont ensuite ajoutées une par une, et les mêmes scénarios sont rejoués après chacune. Un outil n'est conservé que s'il réduit un risque mesuré que les couches précédentes ne couvraient pas déjà.

Tab. 1. Scénarios d'attaque rejoués après chaque couche. Dans tous les cas, l'attaquant exécute du code dans le pod frontend.

ID	Scénario	MITRE ATT&CK
S1	Mouvement latéral depuis le frontend compromis	T1046, T1059
S2	Vol du jeton du compte de service	T1528, T1552.007
S3	Déploiement d'un pod privilégié avec `hostPath`	T1610, T1611
S4	Fuite du mot de passe de la base de données	T1552

Le travail a été réparti en deux parties. Abir Majdi (partie A) a traité les identités et le RBAC, le réseau avec Cilium, l'admission avec Pod Security Admission et Kyverno, ainsi que l'intégration continue. Doha El Merabet (partie B) a traité la gestion des secrets avec Vault et External Secrets Operator, l'audit de l'API, le chiffrement au repos, la détection avec Falco, l'analyse des images avec Trivy et les schémas d'architecture. Chacune a relu et testé le travail de l'autre. Tout le code et les preuves sont publiés dans le dépôt du projet [2].

2. Architecture et environnement de test

2.1. Application et composants

L'application de démonstration comporte trois tiers, chacun isolé dans son propre namespace : un frontend `nginx-unprivileged`, une API `podinfo` et une base PostgreSQL disposant d'un volume persistant de 1 Gi. Un quatrième namespace, `zt-attacker`, héberge un pod qui simule un attaquant interne. Les outils de sécurité sont installés chacun dans un namespace dédié (tableau 2).

Tab. 2. Composants déployés et versions figées dans `versions.md`.

Namespace	Composant
`zt-frontend`, `zt-api`, `zt-data`	Frontend (port 8080), API (port 9898), PostgreSQL 16 (port 5432)
`zt-attacker`	Pod attaquant, image `curl:8.10.1`
`kube-system`	Cilium 1.20.4 et Hubble, CoreDNS
`kyverno`	Kyverno 1.19.2
`vault`, `external-secrets`	Vault 2.0.4 (stockage Raft), External Secrets Operator 2.14.0
`falco`	Falco 0.45.2, Falcosidekick et son interface

Les deux membres du binôme disposent chacun d'un cluster k3s v1.36.5 à un seul nœud : `zt-lab-a` sous Debian 13 et `zt-lab-b` sous Ubuntu 24.04. k3s est installé sans Flannel, sans Traefik et sans son contrôleur de NetworkPolicy intégré, de sorte que Cilium soit à la fois le seul CNI et le seul moteur de politiques réseau. Disposer de deux systèmes différents nous a permis de vérifier que les résultats ne dépendent pas d'une machine particulière.

2.2. Frontières de confiance

Nous avons identifié cinq frontières. Internet n'entre dans l'application que par le frontend. Chaque tier ne communique qu'avec le suivant : le frontend avec l'API sur le port 9898, l'API avec la base sur le port 5432. Les pods applicatifs n'ont aucun besoin de l'API Kubernetes et ne reçoivent donc aucun jeton. Seul le contrôleur d'External Secrets Operator dialogue avec Vault. Enfin, seuls Cilium et Falco franchissent la frontière avec le noyau du nœud, par eBPF, ce qui en fait des composants privilégiés à protéger. La figure 1 montre ce qu'il reste à un attaquant installé dans le frontend une fois toutes les couches en place.

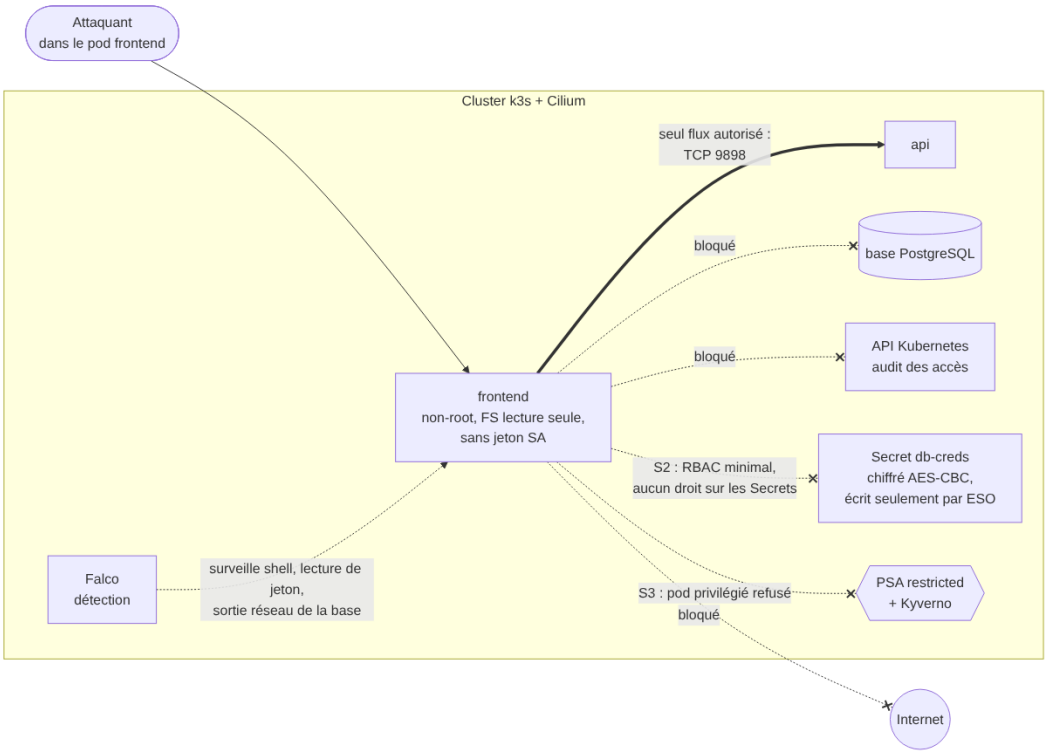


Fig. 1. Surface accessible depuis le pod frontend compromis après mise en place de toutes les couches (schéma issu de `docs/architecture/README.md`).

2.3. Démarche et organisation du travail

Le projet a été conduit par phases, chacune ajoutant une couche de sécurité puis rejouant les quatre scénarios (tableau 3). Les tâches étaient suivies dans un tableau GitHub Projects, à raison d'un ticket par tâche, attribué à son autrice et fermé uniquement lorsque la preuve correspondante était enregistrée dans `docs/evidence/`. Les commits sont signés par clé SSH et un crochet Gitleaks empêche l'envoi de tout secret dans le dépôt.

Tab. 3. Phases du projet et répartition du travail.

Phase	Contenu	Responsable	Cluster
0	Dépôt, modèle de menaces, contrat d'interface (noms, ports, labels)	A. Majdi	
1	Installation de k3s et Cilium, application sans protection, mesure de référence	les deux	a et b
2 à 4	RBAC, NetworkPolicy et Cilium, Pod Security Admission, Kyverno	A. Majdi	a
5	Vault et External Secrets Operator	D. El Merabet	b
6	Audit de l'API, chiffrement au repos, Falco	D. El Merabet	b
7	Intégration de toutes les couches et nouvelle mesure de S1 à S4	les deux	a et b
8 et 9	Conformité, relecture croisée, reconstruction depuis zéro, rapport	les deux	a et b

3. Partie A : identités, réseau et admission

3.1. Comptes de service et RBAC

Dans la configuration initiale, le compte `sa-frontend` disposait du rôle `edit` dans les trois namespaces applicatifs, soit 96 règles dans `zt-data`. Or aucun composant de l'application n'interroge l'API Kubernetes. Le moindre privilège revient donc ici à n'accorder aucune permission : les cinq comptes de service applicatifs n'ont aucun Role, et l'option `automountServiceAccountToken: false` est positionnée sur chaque compte et sur chaque pod.

Après cette étape, `sa-frontend` ne conserve que les 23 règles de découverte accordées par défaut à tout compte authentifié. Nous avons vérifié avec `kubectl auth can-i` que les cinq comptes ne peuvent ni lire les Secrets, ni créer de pods, ni utiliser `pods/exec`, ni créer de RoleBinding, ni recourir aux verbes `escalate`, `bind` ou `impersonate`. Les scénarios S2 et S4 sont ainsi fermés au niveau de l'API.

3.2. Segmentation réseau avec Cilium

Chaque namespace applicatif reçoit une politique de refus par défaut, en entrée comme en sortie, puis une règle autorisant le DNS et une politique par flux légitime. Avant de nous appuyer sur ces règles, nous avons vérifié que le CNI les applique réellement : sans politique, une requête de l'attaquant vers l'API aboutit, alors qu'avec une politique de refus en sortie elle expire (figure 2).

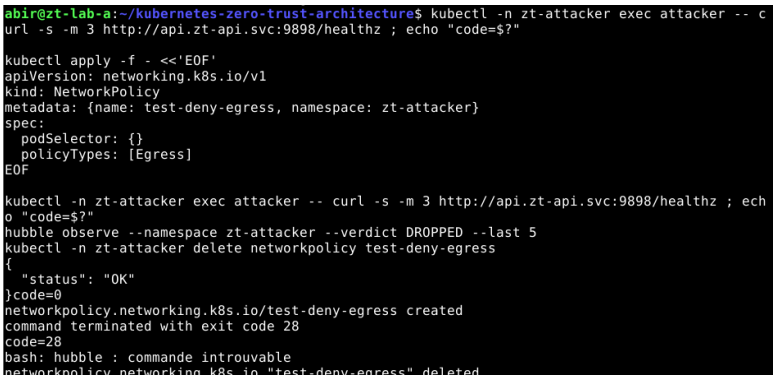


Fig. 2. Vérification de l'application des NetworkPolicy : la même requête renvoie `code=0`, puis `code=28` (délai dépassé) après création de la politique.

Hubble, l'outil d'observation de Cilium, confirme le résultat du côté du frontend. L'extrait suivant, abrégé, provient de la capture du 7 novembre 2026 conservée dans le dépôt :

```
zt-frontend/frontend -> zt-data/database:5432 Policy denied DROPPED (TCP Flags: SYN)
zt-frontend/frontend -> 10.0.2.15:6443 (host) Policy denied DROPPED (TCP Flags: SYN)
zt-frontend/frontend -> 1.1.1.1:443 (world) Policy denied DROPPED (TCP Flags: SYN)
```

Le frontend ne joint donc plus ni la base, ni l'API Kubernetes, ni Internet. Le tableau 4 récapitule les flux autorisés et les flux refusés que nous avons testés.

Tab. 4. Matrice des flux réseau après la phase de segmentation.

Flux	**Résultat**	**Règle**
frontend vers API, TCP 9898	autorisé	`allow-frontend-to-api`
API vers base, TCP 5432	autorisé	`allow-api-to-database`
pods applicatifs vers CoreDNS, port 53	autorisé	`allow-dns-egress`
ESO vers Vault, TCP 8200	autorisé	`network/40-vault-eso.yaml`
Falco vers Falcosidekick	autorisé	`network/50-falco.yaml`
frontend vers base, API Kubernetes ou Internet	refusé	refus par défaut
pod attaquant vers Vault, TCP 8200	refusé	refus par défaut

Une politique CiliumNetworkPolicy de niveau 7 a également été testée (`GET /healthz` autorisé, `GET /version` refusé). Nous l'avons gardée comme démonstration plutôt que de l'étendre à tous les flux, afin de ne pas rendre chaque manifeste dépendant de Cilium.

3.3. Contrôle d'admission

Tous les namespaces applicatifs appliquent le profil `restricted` de Pod Security Admission [4], qui rejette le pod privilégié de S3 avant même l'intervention de Kyverno. Kyverno ne sert donc qu'à exprimer ce que PSA ne sait pas exprimer (tableau 5).

Tab. 5. Politiques Kyverno appliquées aux namespaces `zt-*`.

Politique	Type	Effet
`best-practices-disallow-latest-tag`	validation	Refuse les images sans tag ou marquées `latest`
`best-practices-require-limits`	validation	Impose des limites CPU et mémoire
`supply-chain-restrict-images`	validation	N'admet que les images d'une liste blanche
`defaults-mutate-pod-defaults`	mutation	Ajoute des paramètres de sécurité par défaut
`defaults-generate-default-deny`	génération	Crée un refus par défaut dans chaque nouveau namespace
`secrets-only-from-eso`	validation	Réserve l'écriture des Secrets de `zt-data` à ESO

La relecture croisée par Doha El Merabet a mis au jour deux failles réelles dans ces politiques. La première (R1) permettait de contourner les règles de tag, de limites et de liste blanche en plaçant l'image dans un `initContainer`. La seconde (R10) tenait au fait que Kyverno tolère par défaut les violations existantes : un Secret déjà présent dans `zt-data` restait modifiable par l'administrateur. L'option `allowExistingViolations: false` [3] a corrigé ce point. Les deux corrections ont été retestées sur les deux clusters.

Nous avons aussi mesuré le comportement de Kyverno en cas de panne. Avec un seul réplica, un arrêt propre retire les webhooks et le cluster admet alors tout pod (*fail-open*), tandis qu'un arrêt brutal laisse les webhooks en place et bloque toute création de pod pendant une trentaine de secondes (*fail-closed*). Trois réplicas protégés par un PodDisruptionBudget constitueraient la parade habituelle, que notre machine à un seul nœud ne permettait pas de tester.

Deux choix méritent d'être signalés. Une exception documentée (`PolicyException`) autorise le tag `latest` pour les seuls pods de test nommés `latest-exception-*` dans `zt-attacker`, ce qui montre comment déroger à une règle sans la désactiver. Par ailleurs, les politiques sont passées directement en mode `Enforce`, car le laboratoire ne contenait aucune charge existante. En production, il faudrait d'abord les appliquer en mode `Audit` et examiner les rapports, afin de ne pas bloquer les applications déjà en place.

```
abir@zt-lab-a:~/kubernetes-zero-trust-architecture$ cd ~/kubernetes-zero-trust-architecture
kubectl -n kyverno rollout status deploy/kyverno-admission-controller --timeout=60s
kubectl apply -f admission/ 2>&1 | grep -v Warning
sleep 25
kubectl get cpol
kyverno test admission/tests/ 2>&1 | tail -2
date -Is | tee -a docs/evidence/rebuild/rebuild-$(hostname).md
deployment "kyverno-admission-controller" successfully rolled out
clusterpolicy.kyverno.io/best-practices-disallow-latest-tag created
clusterpolicy.kyverno.io/best-practices-require-limits created
clusterpolicy.kyverno.io/supply-chain-restrict-images created
clusterpolicy.kyverno.io/defaults-mutate-pod-defaults created
clusterrole.rbac.authorization.k8s.io/kyverno:background-controller:networkpolicies unchanged
clusterpolicy.kyverno.io/defaults-generate-default-deny created
policyexception.kyverno.io/allow-latest-tag-for-test created
clusterpolicy.kyverno.io/secrets-only-from-eso created
Warning: kyverno.io/v1 ClusterPolicy is deprecated and will be removed in a future release; m
igrate to ValidatingPolicy, MutatingPolicy, GeneratingPolicy or ImageValidatingPolicy (polici
es.kyverno.io), see https://kyverno.io/docs/guides/migration-to-cel/
NAME                                ADMISSION   BACKGROUND   READY   AGE   MESSAGE
best-practices-disallow-latest-tag   true        true         True    28s   Ready
best-practices-require-limits        true        true         True    27s   Ready
defaults-generate-default-deny       true        true         True    26s   Ready
defaults-mutate-pod-defaults         true        true         True    27s   Ready
secrets-only-from-eso                true        false        True    26s   Ready
supply-chain-restrict-images         true        true         True    27s   Ready
Test Summary: 6 tests passed and 0 tests failed

2026-11-08T00:57:04+02:00
```

Fig. 3. Politiques Kyverno prêtes et six tests `kyverno test` réussis lors de la reconstruction de `zt-lab-a`.

3.4. Mesure automatisée et reproductibilité

Le script `scripts/measure.sh` rejoue les quatre scénarios depuis le pod frontend et enregistre le résultat dans `docs/evidence/` (figure 4). Une chaîne d'intégration continue GitHub Actions exécute Gitleaks sur tout l'historique et les tests Kyverno à chaque envoi. Le 8 novembre 2026, `zt-lab-a` a été entièrement détruit puis reconstruit en suivant uniquement le README, en 47 minutes. Les six critères d'acceptation ont été satisfaits et la mesure obtenue est identique à celle du cluster d'origine.

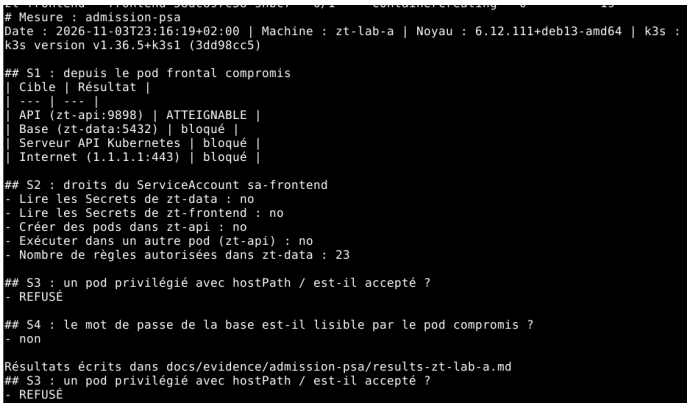


Fig. 4. Sortie de `measure.sh` après l'ajout de PSA : seule l'API reste atteignable et les scénarios S2 à S4 échouent.

4. Partie B : secrets, traçabilité et détection

4.1. Gestion des secrets avec Vault et External Secrets Operator

Dans la configuration initiale, le mot de passe de la base est un Secret Kubernetes créé à la main. Il n'est qu'encodé en base64, n'a pas d'historique et n'est jamais renouvelé. Vault devient la source de vérité unique : External Secrets Operator (ESO) recopie la valeur dans le Secret `db-creds`, que PostgreSQL lit sans modification.

Vault fonctionne avec un stockage Raft persistant, hors mode développement, et le moteur KV v2 conserve les versions successives du secret. Le rôle d'authentification `eso` n'accepte que le compte `eso-vault` du namespace `zt-data`, avec l'audience `vault` et un jeton valable 15 minutes, et la politique associée ne permet de lire que le chemin `secret/data/zt/database/*`. Nous avons préféré ESO au sidecar Vault Agent pour deux raisons : l'application n'a pas à être modifiée, et aucun pod applicatif n'a besoin de communiquer avec Vault. Le tableau 6 compare les trois approches envisagées.

Tab. 6. Comparaison des approches de gestion des secrets.

Critère	**Secret natif**	**Vault et sidecar Agent**	**Vault et ESO (retenu)**
Source de vérité	datastore du cluster	Vault	Vault
Modification de l'application	aucune	annotations, lecture de fichiers	aucune
Flux réseau des pods vers Vault	sans objet	oui, depuis chaque pod	non, seul le contrôleur ESO
Rotation et historique	manuels, aucun	oui	oui (versions KV)
Composants ajoutés	aucun	webhook d'injection et un sidecar par pod	un opérateur
Risque principal	toute identité ayant `get secrets`	webhook d'injection, nombreux clients Vault	ESO peut lire et écrire les Secrets de `zt-data`

Les tests confirment le cloisonnement. Une connexion au rôle `eso` avec un jeton volé de `sa-frontend` ou de `sa-attacker` est refusée, aucun compte applicatif ne peut lire `db-creds`, et le mot de passe n'apparaît nulle part dans le dépôt Git. Le script de rotation change le mot de passe dans PostgreSQL, écrit une nouvelle version dans Vault, force la synchronisation puis vérifie une

connexion réelle à la base (figure 5). Lorsque Vault redémarre scellé, ESO passe en erreur de synchronisation mais l'application continue de fonctionner avec la dernière valeur connue.

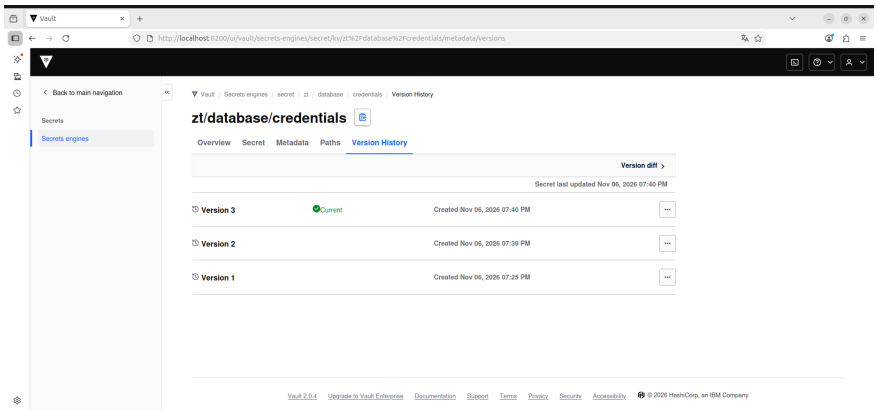


Fig. 5. Historique du secret de la base dans Vault : la copie initiale et deux rotations.

4.2. Audit de l'API et chiffrement au repos

La politique d'audit enregistre les accès aux Secrets au niveau `Metadata`, c'est-à-dire l'auteur, l'action et la date sans jamais le contenu, et conserve le corps complet des modifications sensibles (RBAC, NetworkPolicy, politiques Kyverno). Le journal fait apparaître l'exécution d'une commande dans le frontend ainsi que la tentative de lecture de `db-creds` par l'identité volée, refusée avec le code 403. La valeur du mot de passe n'y figure jamais.

Le chiffrement au repos des Secrets proposé par k3s (AES-CBC) a été activé, puis tous les Secrets existants ont été rechiffrés. Un marqueur de test, lisible en clair dans la base de données du nœud avant l'activation, ne l'est plus ensuite : la valeur stockée commence désormais par `k8s:enc:aescbc:v1:`.

4.3. Détection à l'exécution avec Falco

Falco, avec le pilote `modern_ebpf`, porte cinq règles propres au projet, chacune rattachée à un scénario et à une technique MITRE ATT&CK (tableau 7).

Tab. 7. Règles Falco écrites pour le projet.

Règle	Scénario	ATT&CK	Priorité
Shell lancé dans un conteneur applicatif	S1	T1059	Warning
Lecture du jeton de compte de service	S2	T1528	Warning
Connexion sortante depuis le tier de données	S1, S4	T1041	Notice
Accès interactif aux outils de secrets	S4	T1552	Critical
Lecture du stockage de Vault par un autre processus	S4	T1552.001	Critical

La règle de connexion sortante signale également les tentatives que les NetworkPolicy bloquent, si bien qu'une même attaque est à la fois empêchée par Cilium et détectée par Falco (figure 6). Un faux positif, causé par la sonde de disponibilité de Vault et déclenché toutes les cinq secondes, a été supprimé par une exception documentée.

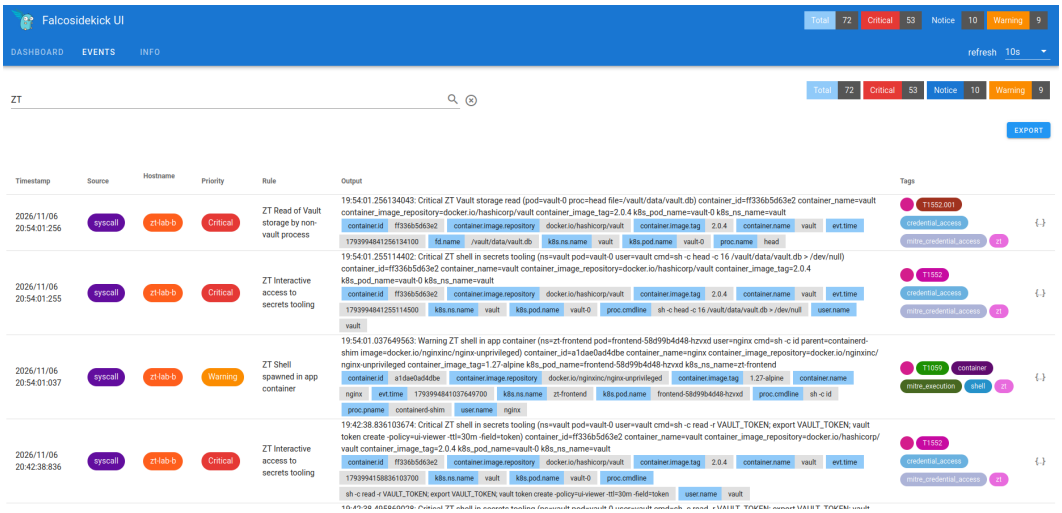


Fig. 6. Alertes du projet dans Falcosidekick, avec leur priorité, le pod concerné et l'étiquette MITRE.

4.4. Analyse des images et seconde reconstruction

Trivy recense 222 vulnérabilités de niveau élevé ou critique dans les quatre images applicatives : 65 pour nginx-unprivileged, 107 pour podinfo, 26 pour postgres et 24 pour curl. Toutes disposent d'une version corrigée et aucune ne provient de la configuration du cluster. Pour nginx-unprivileged, podinfo et curl, la cause est un système de base Alpine ancien, auquel s'ajoutent pour podinfo deux binaires Go compilés avec une version ancienne de Go. Pour postgres, au contraire, le système Alpine 3.24 est récent et sans faille de ce niveau : les 26 résultats viennent tous du binaire Go `gosu` embarqué dans l'image. Nous les avons acceptées dans le cadre du laboratoire, car changer de versions juste avant la remise aurait invalidé les mesures ainsi que la liste blanche de Kyverno. En production, il faudrait mettre à jour les images, les référencer par leur empreinte et ajouter Trivy à la chaîne d'intégration continue pour bloquer toute image présentant une faille critique corrigeable.

Le 8 novembre 2026, `zt-lab-b` a lui aussi été reconstruit à partir du README, en 55 minutes. Les six critères ont été satisfaits et la mesure obtenue est identique à celle de `zt-lab-a`. Cette seconde reconstruction, menée sur un autre système et par l'autre membre du binôme, a révélé d'autres manques que la première : la liste des outils à installer au préalable, le répertoire `/etc/cni/net.d` laissé en place par la désinstallation de k3s, l'attente de l'enregistrement du nœud avant les vérifications, et un délai d'attente trop court dans le script de descellement de Vault. Tous ont été corrigés depuis.

5. Résultats et évaluation

5.1. Réduction du rayon d'action

Le tableau 8 résume ce que peut faire le pod frontend compromis à chaque étape. Le RBAC ferme l'accès aux Secrets et la création de pods, les NetworkPolicy coupent les trois cibles réseau illégitimes, et PSA rejette le pod dangereux. Vault, ESO et Falco ne modifient pas ces colonnes, car ils agissent sur d'autres risques : qui peut obtenir le secret, son renouvellement, et la visibilité de l'attaque. La même mesure, rejouée sur `zt-lab-b`, donne un résultat identique ligne par ligne.

Tab. 8. Rayon d'action du frontend compromis après chaque couche (source : `docs/evidence/blast-radius.md`).

Étape	API	Base	API K8s	Internet	Secrets	Pods	S3	S4
Initiale	oui	oui	oui	oui	oui	oui	accepté	lisible
+ RBAC	oui	oui	oui	oui	non	non	accepté	non
+ NetworkPolicy	oui	non	non	non	non	non	accepté	non
+ PSA	oui	non	non	non	non	non	refusé	non
+ Kyverno	oui	non	non	non	non	non	refusé	non
Intégration	oui	non	non	non	non	non	refusé	non

5.2. Conformité et coût

Le banc d'essai kube-bench (référentiel CIS `k3s-cis-1.9`) donne 59 contrôles réussis et 4 en échec après deux corrections. Les quatre échecs restants ont été analysés : deux faux positifs liés au chemin du CNI, un contrôle non applicable puisque k3s utilise SQLite et non etcd, et un écart justifié, les composants système ayant besoin de leur jeton. Sur `zt-lab-b`, le résultat passe de 57 réussis et 6 échecs à 59 réussis et 4 échecs. Kubescape (référentiels NSA et MITRE) a relevé un seul défaut sur nos workloads, le contrôle C-0013 lié à un `runAsGroup` manquant, qui a été corrigé. Les 46 ressources encore signalées appartiennent toutes à des composants tiers ou à k3s.

Ce niveau de protection a un coût. Mesurée au repos sur `zt-lab-b` avec `kubectl top`, l'application occupe 35 MiB de mémoire, alors que Kyverno en occupe 306, Cilium et Hubble 303, ESO 124, Falco et ses composants 116, et Vault 46. Les outils de sécurité consomment donc environ 26 fois plus de mémoire que l'application qu'ils protègent. Sur un cluster de production, ce coût serait partagé entre de nombreuses applications, mais il confirme que chaque outil doit justifier sa présence.

5.3. Sécurité du stockage

La base PostgreSQL utilise un volume persistant de 1 Gi fourni par le provisionneur `local-path` de k3s, qui crée un répertoire sur le nœud. Le pod tourne sans droits root, le groupe `postgres` (`fsGroup: 70`) possède le volume, et aucun autre pod ne le monte. Dans chaque pod applicatif, le système de fichiers racine est en lecture seule : seuls des volumes `emptyDir`, qui disparaissent avec le pod, sont accessibles en écriture. Les Secrets sont chiffrés au repos et les données de Vault sont protégées par sa propre clé de chiffrement.

Le volume `hostPath` est la configuration la plus dangereuse, car il monte un répertoire du nœud dans le conteneur. Avec `hostPath: /`, comme dans le scénario S3, un conteneur peut lire `/etc/shadow` et les identifiants de k3s, y compris la clé de chiffrement des Secrets, puis prendre le contrôle du nœud. Le profil `restricted` de PSA n'autorise que des types de volumes sûrs (`configMap`, `emptyDir`, `persistentVolumeClaim`, `projected`, `secret`, entre autres) et rejette donc ce pod.

5.4. Limites

La principale limite tient au nœud unique : un attaquant qui compromet le nœud contourne l'ensemble des contrôles. Le flux légitime du frontend vers l'API reste par ailleurs ouvert et pourrait être détourné. Plusieurs raccourcis propres au laboratoire subsistent : Vault fonctionne sans TLS et avec une seule clé de descellement, la clé de chiffrement au repos et les journaux d'audit résident sur le même nœud, et Kyverno ne compte qu'un réplica. Les images applicatives ne sont ni à jour ni référencées par leur empreinte. Le DNS reste un canal d'exfiltration possible et le trafic entre pods n'est pas chiffré. Enfin, Falco détecte sans bloquer, et un Falco arrêté constituerait un angle

mort silencieux.

6. Réponses aux questions d'analyse du sujet

Quels sont les composants les plus sensibles ? Par ordre d'importance : le serveur d'API et sa base de données, qui contrôlent tout le cluster ; Vault et sa clé de descellement, qui protègent tous les secrets applicatifs ; le contrôleur ESO, qui peut écrire tous les Secrets de `zt-data` ; enfin Cilium et Falco, qui ont accès au noyau. Côté application, la base de données est la cible la plus précieuse.

Quelles permissions chaque composant nécessite-t-il ? Les workloads applicatifs n'ont besoin d'aucune permission sur l'API Kubernetes et ne reçoivent aucun jeton. ESO doit lire et écrire les Secrets de `zt-data` et demander un jeton pour `eso-vault`. Vault a besoin du rôle `system:auth-delegator` pour valider les jetons, et Falco d'un accès au socket du runtime et au noyau.

Que peut atteindre un attaquant après la compromission d'un pod ? Dans la configuration initiale, tous les services, l'API Kubernetes, Internet et tous les secrets de `zt-data`. Avec toutes les couches, uniquement l'API sur le port 9898, sans jeton pour parler à Kubernetes et sans possibilité de se connecter à Vault, même avec un jeton volé.

Qu'est-ce qui empêche ou limite le mouvement latéral ? Le refus réseau par défaut appliqué par Cilium, l'absence de jeton et le RBAC minimal, les rôles Vault liés à une seule identité, ainsi que PSA et Kyverno contre l'évasion vers le nœud. Falco et le journal d'audit n'empêchent pas le mouvement, mais le rendent visible.

Quels sont les risques d'une mauvaise configuration des volumes ? Un `hostPath` expose le système de fichiers du nœud, un système de fichiers racine inscriptible permet de déposer des outils, un volume partagé entre plusieurs pods expose les données à une autre charge, et une base non chiffrée laisse fuir les Secrets à partir d'une simple sauvegarde.

Quelles sont les limites des contrôles ? Elles sont détaillées dans la section 5.4. La principale est l'exécution sur un seul nœud : la compromission de ce nœud annule tous les contrôles.

7. Discussion : justification de chaque outil

Pour répondre à la question centrale, nous avons examiné chaque outil selon quatre critères : le risque qu'il réduit de façon mesurée, ce que les couches précédentes couvraient déjà, la surface d'attaque qu'il ajoute, et le comportement du cluster s'il tombe en panne (tableau 9).

Tab. 9. Justification de chaque mécanisme de sécurité retenu.

Mécanisme	**Risque réduit (mesuré)**	**Déjà couvert par**	**Surface ajoutée**	**En cas de panne**
RBAC sans jeton (natif)	S2 et S4 : de 96 à 23 règles, aucun accès aux Secrets	rien, première couche	aucune	sans objet
NetworkPolicy (natif)	S1 : de 4 cibles atteignables à 1	rien au niveau L3/L4	aucune	dépend du CNI
Cilium et Hubble	Applique les NetworkPolicy, ajoute L7 et la visibilité des flux	en partie (tout CNI applique L3/L4)	agent eBPF privilégié	plus de réseau pour les pods
PSA (natif)	S3 : pod privilégié et `hostPath` refusés	rien	aucune	sans objet, intégré à l'API
Kyverno	Ce que PSA ne sait pas exprimer : tags, limites, images, refus par défaut généré	en partie (S3 déjà couvert par PSA)	webhook avec droits sur tout le cluster	permissif après un arrêt propre, bloquant 30 s après un crash
Vault et ESO	S4 : secret possédé, versionné, renouvelé, inutilisable avec un jeton volé	en partie (le RBAC le cache déjà)	Vault, clé de descellement, contrôleur ESO	Secret existant conservé, application intacte
Audit et chiffrement (natifs)	Traçabilité ; S4 à partir d'une sauvegarde volée	rien	volume de journaux, clé sur le nœud	perte de traçabilité
Falco	Visibilité de S1, S2 et S4, même bloqués	rien (détection)	pod privilégié, accès noyau	angle mort silencieux

Nous avons aussi choisi de ne pas ajouter certains mécanismes. Les règles de niveau 7 de Cilium sont restées une démonstration, car les NetworkPolicy ferment déjà S1 et leur généralisation lierait chaque manifeste à Cilium. Kyverno ne reproduit pas les règles de PSA. Enfin, nous n'avons pas déployé de maillage de services : son principal apport, le chiffrement mutuel mTLS, n'est mesuré par aucun de nos scénarios, et Cilium peut fournir un chiffrement transparent à moindre coût.

8. Conclusion

Notre réponse à la question posée tient en une règle : commencer par les mécanismes natifs de Kubernetes, puis n'ajouter un outil que lorsqu'un risque mesuré reste ouvert. Le RBAC sans jeton, les NetworkPolicy et Pod Security Admission ne coûtent presque rien et ferment à eux seuls les quatre scénarios. Le pod frontend compromis passe ainsi de quatre cibles atteignables à une seule, l'API, sur son flux légitime. Chaque outil ajouté ensuite répond à un risque que le natif laissait ouvert, comme le montre la section précédente, et nous avons renoncé à ceux dont le bénéfice n'était pas mesurable dans nos scénarios.

La démarche elle-même s'est révélée utile. La relecture croisée a mis au jour de véritables failles, toutes corrigées et retestées, et les deux reconstructions indépendantes, sur deux systèmes et par deux personnes, ont abouti à la même mesure. Pour une mise en production, les priorités seraient de passer à plusieurs nœuds, de faire tourner Kyverno sur trois réplicas, de déployer Vault avec TLS et un descellement automatique, et d'utiliser des images à jour référencées par leur empreinte.

Le dépôt, en version `v1.0.2`, contient l'ensemble du code et des preuves citées. La vidéo de démonstration est disponible à l'adresse suivante : `https://drive.google.com/file/d/1gj5EWyhtxk6pGZi1-1RBWAKOx2ShKWZ_/view?usp=drive_link`.

Références

[1] S. Rose, O. Borchert, S. Mitchell, S. Connelly, *Zero Trust Architecture*, NIST Special Publication 800-207, août 2020.

[2] A. Majdi, D. El Merabet, *kubernetes-zero-trust-architecture*, dépôt GitHub, version v1.0.2, 2026. `https://github.com/penelopeeckhar/kubernetes-zero-trust-architecture`

[3] Kyverno, *Validate Rules*, documentation officielle. `https://kyverno.io/docs/policy-types/cluster-policy/validate/`

[4] The Kubernetes Authors, *Pod Security Standards* et *Network Policies*, documentation officielle de Kubernetes. `https://kubernetes.io/docs/`